BENCHMARK REPORT

PagedServe

A from-scratch LLM inference engine with continuous batching and a paged KV cache, built for a 4 GB consumer GPU

Subhajit Sarkar | github.com/Subhajit281/pagedserve | October 2026

Headline results

137.20 tok/s
Integrated Triton backend, 8 concurrent requests

2.34x
Token throughput vs. PyTorch backend at concurrency 8 (58.65 to 137.20 tok/s)

31.9x
Lower median latency for the Triton paged-decode kernel (8.104 to 0.254 ms)

1.51x
Throughput gain from continuous batching, 1 to 8 requests (23.40 to 35.34 tok/s)

~1.0 GiB
Peak GPU memory in every serving run, on a 4 GB device

128 passed
Automated tests (1 expected failure: known Triton parity issue)

All figures were measured on the single development machine below. Results are specific to the stated workloads; scope notes accompany each table.

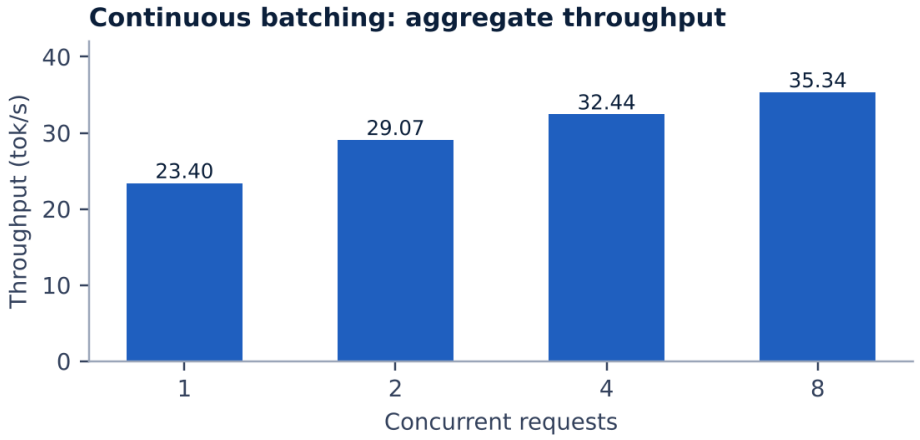
Test environment

Property	Configuration
GPU	NVIDIA RTX 3050 Laptop GPU, 4 GB VRAM
CPU	Intel Core i5-12500H
System RAM	16 GB
Runtime	WSL2, Ubuntu 24.04; Python, PyTorch, FastAPI, Triton
Model	Qwen2.5-0.5B-Instruct
Precision	BF16 for primary experiments; FP32 for selected numerical checks

1. Continuous batching throughput

Iteration-level scheduling over the paged KV cache; measured at increasing numbers of concurrent requests.

Concurrency	Throughput	Speedup vs. concurrency 1	Peak GPU memory
1	23.40 tok/s	1.00x	984.75 MiB
2	29.07 tok/s	1.24x	989.28 MiB
4	32.44 tok/s	1.39x	1,002.06 MiB
8	**35.34 tok/s**	**1.51x**	**1,002.06 MiB**

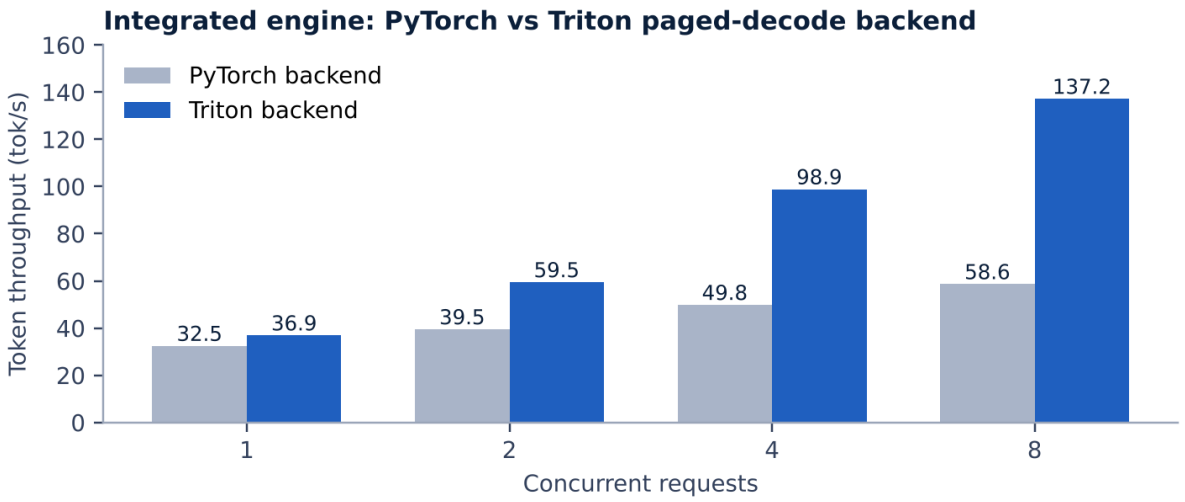


Scope: scaling is sublinear and applies to this workload only. The run also verified request completion, cache cleanup and decode-batch consistency.

2. Integrated engine: PyTorch vs. Triton paged-decode backend

Full engine workload including scheduling, prefill, decode and cache management (not only the attention kernel).

Concurrency	Backend	Request latency	Requests/s	Tokens/s	Peak GPU memory
1	PyTorch	0.986 s	1.01	32.47	1,008.8 MiB
1	**Triton**	**0.867 s**	**1.15**	**36.91**	**1,008.8 MiB**
2	PyTorch	1.620 s	1.23	39.51	1,008.8 MiB
2	**Triton**	**1.076 s**	**1.86**	**59.46**	**1,008.8 MiB**
4	PyTorch	2.568 s	1.56	49.84	1,008.8 MiB
4	**Triton**	**1.295 s**	**3.09**	**98.88**	**1,008.8 MiB**
8	PyTorch	4.365 s	1.83	58.65	1,008.8 MiB
8	**Triton**	**1.866 s**	**4.29**	**137.20**	**1,013.1 MiB**



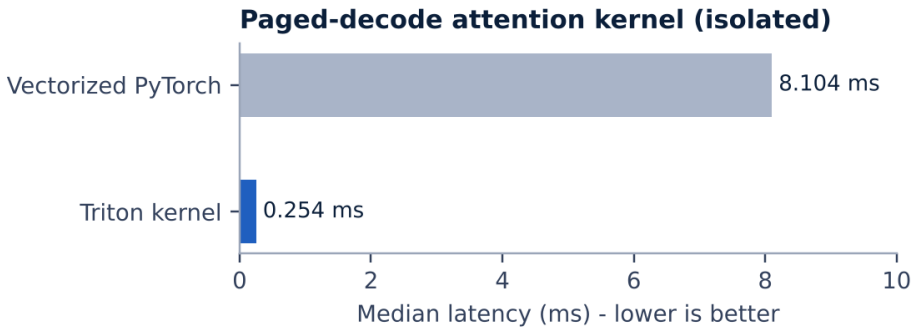
Concurrency	Throughput gain (Triton / PyTorch)	Request-latency reduction
1	1.14x	12.1%
2	1.50x	33.6%
4	1.98x	49.6%
8	**2.34x**	**57.3%**

Scope: the Triton advantage grows with concurrency. Exact output parity with the PyTorch path held at concurrency 2 but not at 1, 4 and 8, so the Triton backend is documented as experimental rather than a verified drop-in replacement.

3. Triton paged-decode kernel microbenchmark

Isolated attention kernel; excludes model projections and the generation loop.

Implementation	Median latency	Mean latency
Vectorized PyTorch reference	8.104 ms	9.579 ms
Triton kernel	**0.254 ms**	**0.270 ms**



Result: 31.9x lower median latency. The kernel matrix benchmark passed numerical checks for all tested cases, with a maximum absolute error of about 0.01074. This is a kernel-level result, not an end-to-end speedup.

4. Single-request latency and sequential baseline

Prompt-length sensitivity of PagedServe, alongside a sequential Hugging Face reference (5 repeats, max_new_tokens=32).

PagedServe, single request

Prompt	Throughput	Prefill latency	Time to first token	Time / output token	Peak GPU memory
Short	31.05 tok/s	55.23 ms	56.35 ms	31.32 ms	984.75 MiB
Medium	24.14 tok/s	202.28 ms	202.43 ms	36.13 ms	988.81 MiB
Long	16.77 tok/s	610.99 ms	611.16 ms	41.72 ms	997.30 MiB

Sequential Hugging Face baseline

Prompt	Prompt tokens	Mean latency	Median latency	Throughput	Peak GPU memory
Short	5	825.0 ms	831.3 ms	38.79 tok/s	961.33 MiB
Medium	19	807.0 ms	818.4 ms	39.65 tok/s	961.50 MiB
Long	48	828.1 ms	835.8 ms	38.64 tok/s	961.85 MiB

Scope: the two implementations use different execution paths, so this is a reference comparison rather than a strict like-for-like test. Single-request PagedServe performance is dominated by prefill on longer prompts; its advantages appear under concurrency (Sections 1 and 2).

5. Static batching baseline

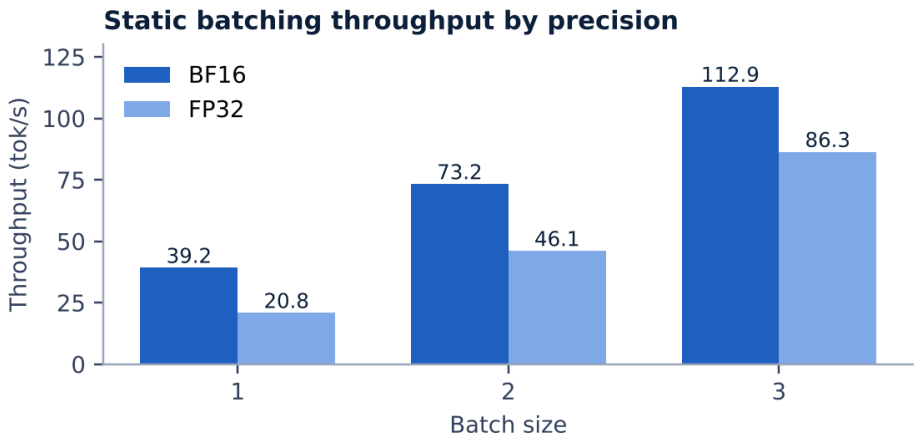
Batched generation without iteration-level scheduling, in BF16 (5 repeats) and FP32 (3 repeats).

BF16

Batch size	Throughput	Per-request latency	Peak GPU memory	Token parity
1	39.18 tok/s	816.82 ms	961.33 MiB	Passed
2	73.25 tok/s	436.83 ms	964.69 MiB	Diverged
3	112.89 tok/s	283.47 ms	968.93 MiB	Diverged

FP32

Batch size	Throughput	Per-request latency	Peak GPU memory	Token parity
1	20.84 tok/s	1,535.60 ms	1,897.15 MiB	Passed
2	46.07 tok/s	694.60 ms	1,902.63 MiB	Passed
3	86.33 tok/s	370.66 ms	1,907.67 MiB	Passed



Scope: FP32 restores token parity on the tested cases at roughly 2x the memory and higher latency, illustrating the numerical-stability trade-off. Parity is not established for every prompt or configuration.

6. KV-cache sizing and fragmentation

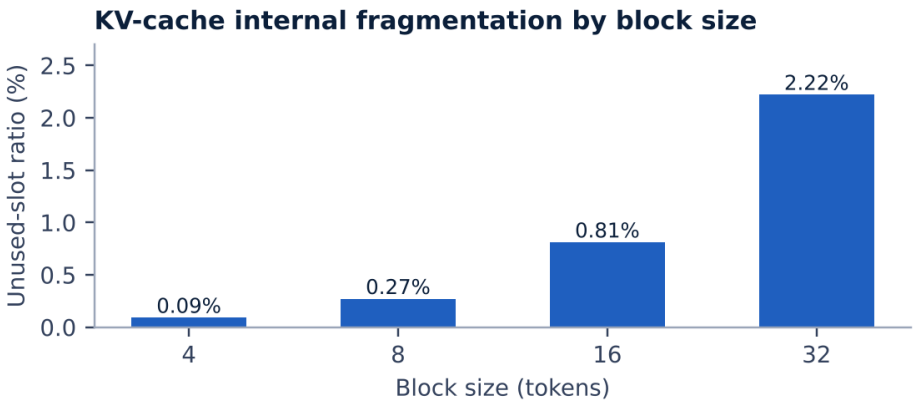
Analytical allocation analysis (not an end-to-end allocator benchmark).

Cache footprint (Qwen2.5-0.5B, BF16)

Quantity	Value
KV-cache footprint per token	~12 KiB
Block size (default)	16 tokens
KV memory per block	~192 KiB
Configured pool	128 blocks = ~24 MiB = 2,048 token slots

Block-size trade-off on the analytical workload

Block size	Blocks required	Reserved token slots	Unused slots	Unused ratio	Est. KV memory
4	1,104	4,416	4	0.09%	51.750 MiB
8	553	4,424	12	0.27%	51.844 MiB
16	**278**	**4,448**	**36**	**0.81%**	**52.125 MiB**
32	141	4,512	100	2.22%	52.875 MiB



Scope: smaller blocks waste fewer token slots at the cost of more block-table bookkeeping. Block-based allocation keeps internal fragmentation below 1% at the default 16-token block size.

7. Validation summary

Check	Result
Full automated test suite	128 passed, 1 expected failure (known full-model Triton parity issue), 1 warning
Coverage	API, request handling, block allocation, KV cache, scheduler, paged attention, model integration, Triton components
Token-sequence parity	Verified on three validation prompts; all KV-cache allocations released afterward
Triton kernel numerical check	Passed on all tested cases; max absolute error about 0.01074

Reproducibility: benchmark scripts live under `scripts/` and recorded result artifacts under `bench/results/` in the repository. Performance varies with prompt length, output length, concurrency, precision, warm-up and hardware.